

Lecture 8 (Part 1) — Abstract Classes

- Abstraction in OOP
- Why abstract classes are needed
- Python's ABC and @abstractmethod

What is Abstraction?

- Hiding internal complexity and exposing only what is necessary.
- In business systems: users see a clean interface, not implementation details.
- Example: You use a 'pay' button, not the full tax + fee calculation logic.

Why Do We Need Abstract Classes?

- Normal classes allow creating objects that may be incomplete or meaningless.
- Abstract classes define a common template that all subclasses must follow.
- They enforce that certain methods **MUST** exist (e.g., `calculate_pay`, `process_payment`).
- This keeps large systems consistent and less error-prone.

Python Syntax — Abstract Class

- Use ABC (Abstract Base Class) and @abstractmethod from abc module.
- Cannot create an object directly from an abstract class.

```
from abc import ABC, abstractmethod
```

```
class Payment(ABC):  
    @abstractmethod  
    def process(self, amount):  
        pass
```

Business Example — Payment System

- We want every payment type to implement process().
- Abstract class Payment defines the common interface.

```
class CardPayment(Payment):
    def process(self, amount):
        print(f"Processing card payment of {amount}")

class WalletPayment(Payment):
    def process(self, amount):
        print(f"Processing wallet payment of {amount}")

p1 = CardPayment()
p1.process(1000)
```

Shape Example — area() as Abstract Method

- We never want a raw Shape object, only real shapes like Circle, Rectangle.
- Shape defines the promise: every shape must implement area().

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, r):
        self.r = r
    def area(self):
        return 3.14 * self.r * self.r
```

Key Rules of Abstract Classes (Python)

- You cannot instantiate an abstract class (it has at least one abstract method).
- All abstract methods must be overridden in concrete subclasses.
- Abstract classes can still have normal (concrete) methods.
- They are mainly about enforcing a common design/contract.

When Should You Use an Abstract Class?

- When multiple subclasses share the same interface but different implementations.
- When you want to force every child to implement some critical methods.
- Examples: `Employee.calculate_pay()`, `Account.withdraw()`, `Report.generate()`.

Practice — Abstract Class Design

- Design an abstract class `Vehicle` with abstract method `start_engine()`.
- Create two concrete classes: `Car` and `Bike` that implement `start_engine()` differently.
- Add one more abstract method `get_max_speed()` and implement it in both subclasses.

Summary — Abstract Classes

- Abstraction hides complexity and shows only essentials.
- Abstract classes define a template and enforce required methods.
- They are crucial in large business and fintech systems for consistency.
- Next: Exception Handling — protecting these systems from runtime errors.